

EVE310 Lab 4

September 14, 2023

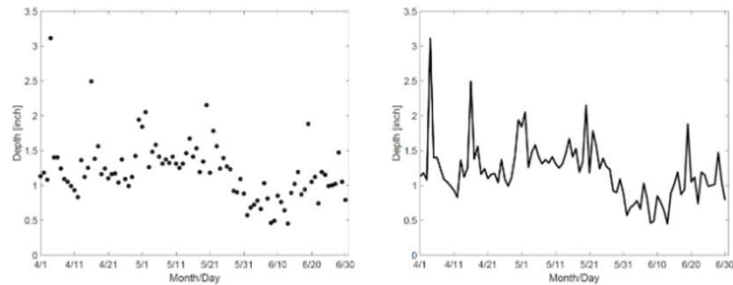
Learning objectives

- Use Matplotlib and Scipy package
- Convert to datetime format
- Creating graphics of data
 - Plot
 - Boxplot
 - Histogram
- Save Plots to PDF or Image
- Initial Exploratory Data Analysis

Remember back to Lecture 3

Common graphical methods for analysis of a univariate dataset

1. Time series plots

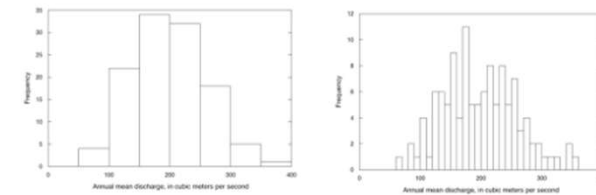


15

Common graphical methods for analysis of a univariate dataset

2. Histograms

- Histograms illustrate the **central tendency**, **variability**, and **symmetry** of the dataset.
- If the sample was infinitely large then they would converge to being the probability density function of the population
- **Note:** depends on the number of categories selected for the plot

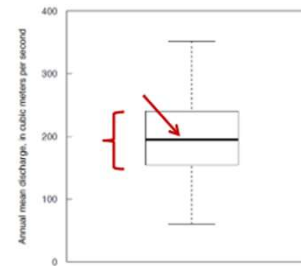


17

Common graphical methods for analysis of a univariate dataset

2. Boxplots – summarize the distribution of a dataset:

- The **line** in the box represents the **median** of the data
- The box **height** is the variation of spread (**interquartile range**), i.e., 50% of the data



20

Start by importing the packages

2 New Packages: Matplotlib and SciPy

Matplotlib is used to create graphics from data

Scipy is used for scientific calculations and statistics

```
#Import packages  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import scipy.stats as sc  
  
#Additional command to tell jupyter notebook to show the plots directly underneath the code cells  
%matplotlib inline
```

Use this line any time you are making a plot

Use this line only when using Jupyter Notebook, which tells the computer to show figures directly underneath code blocks

A Quick Review of our Packages

Package

General Purpose

NumPy

Algebraic Calculations

Pandas

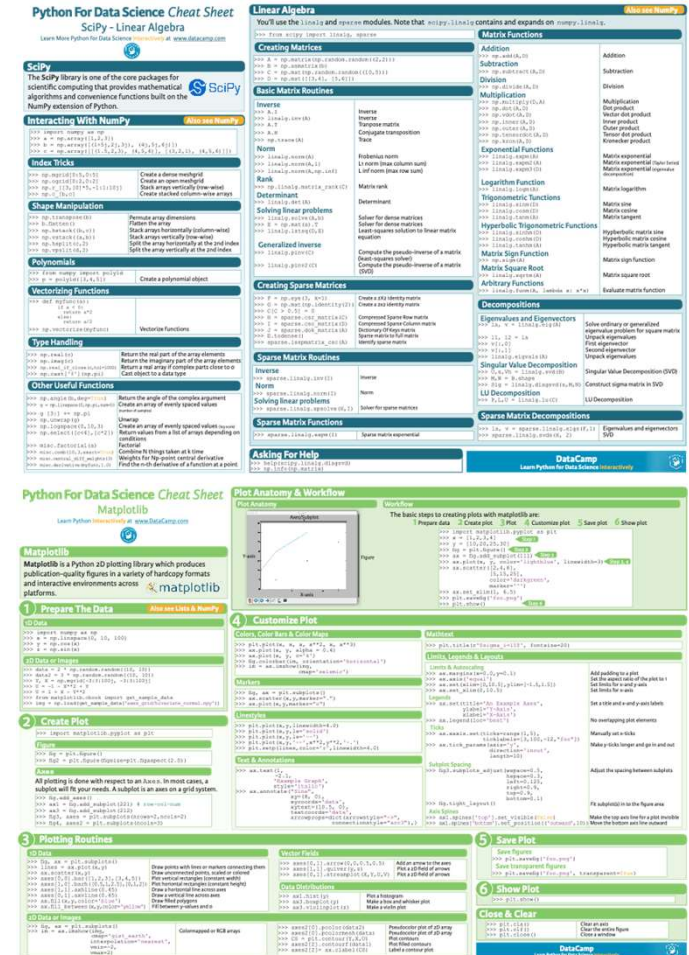
Databases, real-world imported data (ex: sensor measurements)

Matplotlib

Creating plots

SciPy

Advanced statistical calculations



Easy way to refer to an entire column of a dataframe

- loc and iloc allow you to index a dataframe similar to how you would index an array
- You can specify a specific row you are interested in, or use a : to extract all rows
- Another, simpler way to extract all rows of a column is to put the column name in square brackets after the name of the dataframe

The following lines are all equivalent:

```
#Extract the entire allDate column using iloc:  
date_column=water_df.iloc[:,0]  
  
#Extract the entire allDate column using loc:  
date_column=water_df.loc[:, 'allDate']  
  
#Extract the entire allDate column using square brackets instead of loc or iloc  
date_column=water_df['allDate']
```



The following slides will use this notation to refer to an entire column of a dataframe

Begin by importing the data and checking the head and info

```
#Import the data into python as a dataframe from the csv file
water_df = pd.read_csv('JES_Water_Lab4_Tutorial.csv')

#Take a look at the first 5 rows of the dataframe using the head function
water_df.head(5)
```

	allDate	prevDay	prevWeek	avgTemp	wF	MonMed	classSession	dayType	season
0	5/1/2015 0:00	132.867954	83.316913	69.5	94.711003	44.970692	1	wk	Spring
1	5/2/2015 0:00	94.711003	83.316913	70.5	42.131633	44.970692	1	wknd	Spring
2	5/3/2015 0:00	42.131633	83.316913	71.0	61.512942	44.970692	1	wknd	Spring
3	5/4/2015 0:00	61.512942	83.316913	75.0	132.073017	44.970692	1	wk	Spring
4	5/5/2015 0:00	132.073017	83.316913	74.5	132.073017	44.970692	1	wk	Spring

```
1 #Show the metadata of the dataframe
2 water_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 762 entries, 0 to 761
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   allDate          762 non-null    object
1   prevDay          762 non-null    float64
2   prevWeek         762 non-null    float64
3   avgTemp          762 non-null    float64
4   wF               762 non-null    float64
5   MonMed           762 non-null    float64
6   classSession     762 non-null    int64
7   dayType          762 non-null    object
8   season           762 non-null    object
dtypes: float64(5), int64(1), object(3)
memory usage: 53.7+ KB
```

We can see that the allDate column clearly indicates a date, but the computer sees it as just text, as indicated by the type of “object” shown in the info.

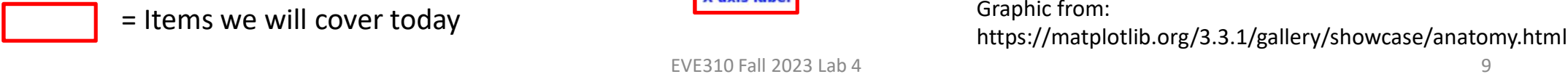
Convert allDate column to Datetime Format

```
1 #Convert allDate column to datetimes
2 water_df['allDate'] = pd.to_datetime(water_df['allDate'])
3
4 #Check the info again to make sure it worked
5 water_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 122 entries, 0 to 121
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   allDate          122 non-null    datetime64[ns]
1   prevDay          120 non-null    float64
2   prevWeek         120 non-null    float64
3   avgTemp          122 non-null    float64
4   wF               120 non-null    float64
5   MonMed           122 non-null    float64
6   classSession     122 non-null    int64
7   dayType          122 non-null    object
8   season           122 non-null    object
dtypes: datetime64[ns](1), float64(5), int64(1), object(2)
memory usage: 8.7+ KB
```

Success!

Let's Get Plotting!



Elements of a figure

1. Create the figure handle

```
#Create a New Figure  
fig1=plt.figure()
```

2. Display the data

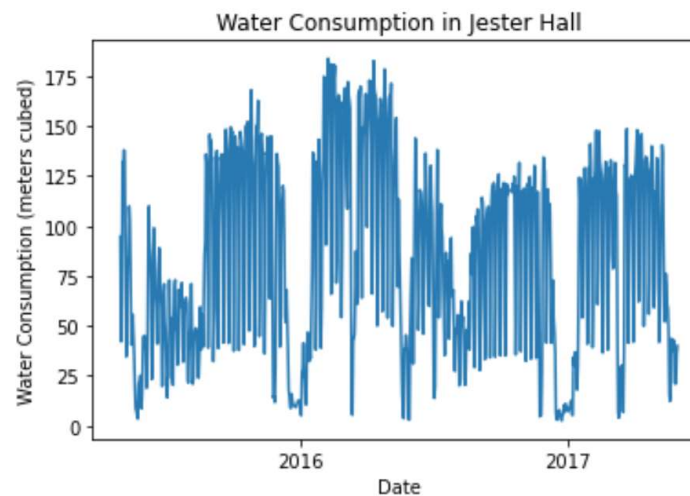
```
#Create the plot with the date on the X axis and water consumption on the Y axis  
plt.plot(water_df['allDate'],water_df['wF'])
```

3. Adjust display options

```
#Format X axis. We'll use this command only when we have dates on the X axis.  
#It autoformats the dates to clean it up and make the axis look better  
plt.gcf().autofmt_xdate()
```

4. Add labels

```
#Label the axes and add a title. Always remember to label your axes and include units  
plt.xlabel('Date')  
plt.ylabel('Water Consumption (meters cubed)')  
plt.title('Water Consumption in Jester Hall')
```



→ Default style: a blue line connecting each point

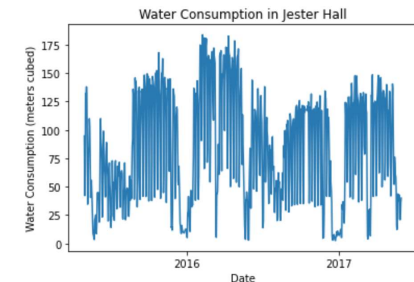
Variations within the plot function

- Inputs should be a column vector (one column) of the same size

```
plt.plot(water_df['allDate'], water_df['wF'])
```

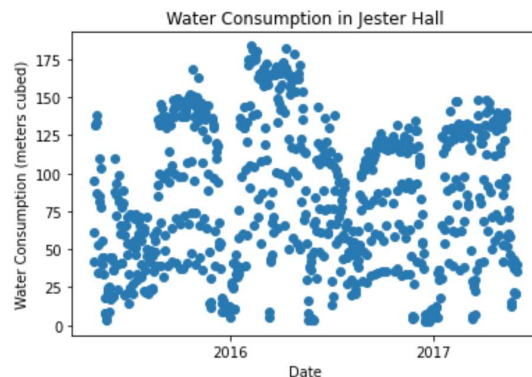
Plotted on the x axis

Plotted on the y axis



- Add additional specifications to show dots for each data point instead of connected by a line

```
plt.plot(water_df['allDate'], water_df['wF'], marker='o', ls='')
```



Specify markers
as dots

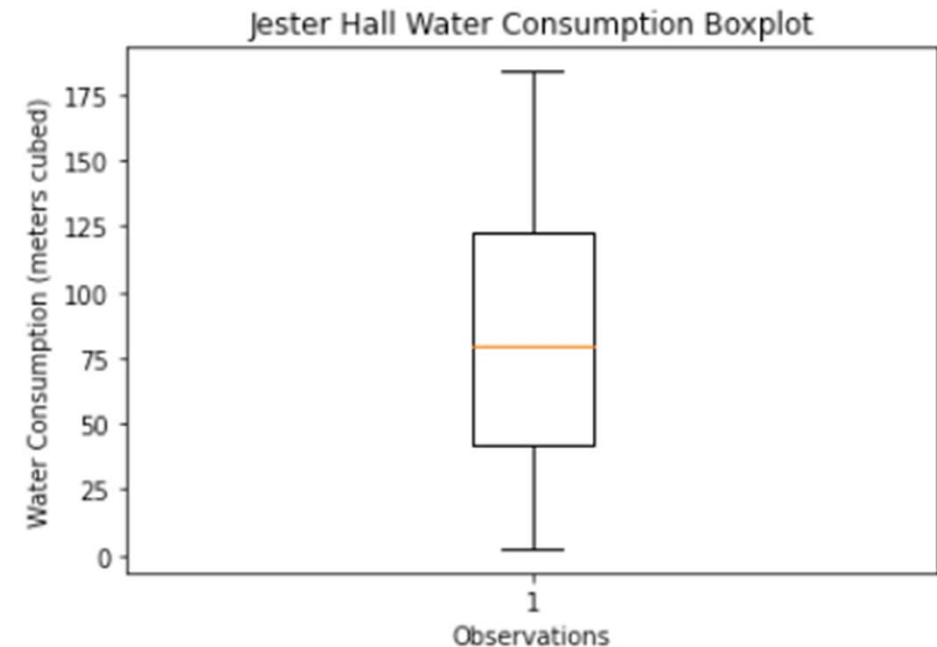
Do not draw a line
between points

→ Show each point as a circle without
a line connecting each point

Boxplot

- Matplotlib requires input to be a NumPy array without NaN values
- Use plt.boxplot

```
#Convert the wF column of the dataframe to an array  
water_a=water_df[ 'wF' ].to_numpy()  
  
#Create a figure  
fig3=plt.figure()  
  
#Create the boxplot  
plt.boxplot(water_a)  
  
#Add labels and title  
plt.xlabel('Observations')  
plt.ylabel('Water Consumption (meters cubed)')  
plt.title('Jester Hall Water Consumption Boxplot')
```



Histogram

- Requires input to be a numpy array without NaN values
- Use plt.hist
- Default is for the number of bins to be automatically selected. We can specify the number of bins

```
#Convert the wF column of the dataframe to an array  
water_a=water_df['wF'].to_numpy()
```

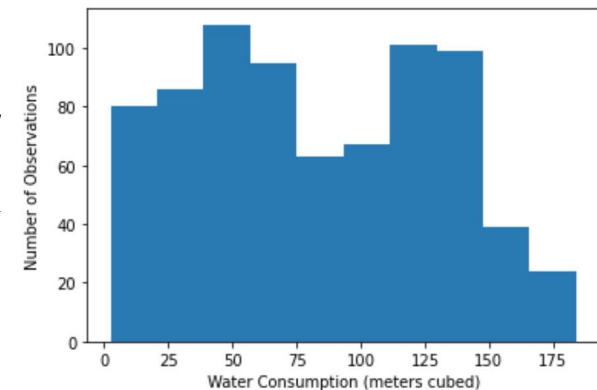
```
#Create a figure  
fig4=plt.figure()
```

```
#Create the histogram  
plt.hist(water_a)
```

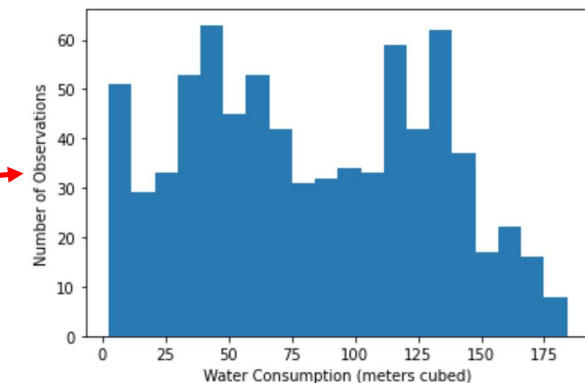
```
plt.xlabel('Water Consumption (meters cubed)')  
plt.ylabel('Number of Observations')
```

```
#Create the histogram and specify 20 bins  
plt.hist(water_a,20)
```

**Bin #
automatically
selected**



**Bin # of 20
specified**



Controlling Font Size

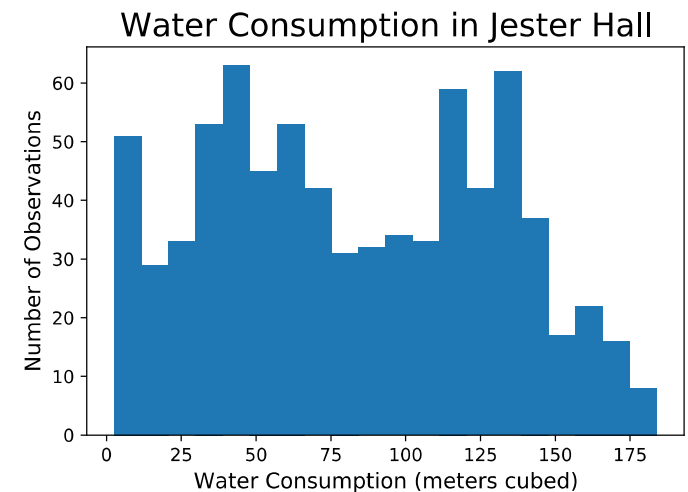
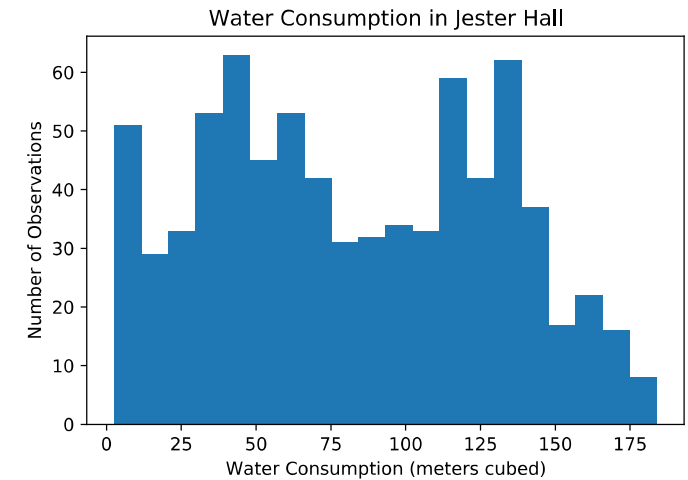
- If not specified, default font sizes will be used which might not be big enough/appropriate for your specific project
- Specify the font size when creating labels and title

```
#Create a figure
fig6=plt.figure()

#Create the histogram and specify 20 bins
plt.hist(water_a,20)

#Add labels and title with specified font sizes
plt.xlabel('Water Consumption (meters cubed)',fontsize=12)
plt.ylabel('Number of Observations',fontsize=12)
plt.title('Water Consumption in Jester Hall',fontsize=18)
```

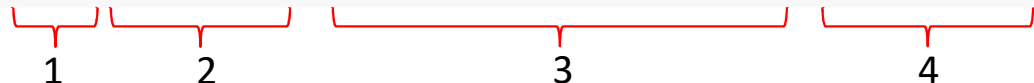
Default font size



Save figures to PDF or PNG file

1. Handle of figure that is already created
2. savefig function
3. String of the file name, ending with the file format
4. Resolution (dpi means dot per inch). It's fine to always use 600 (default if not specified is 100).

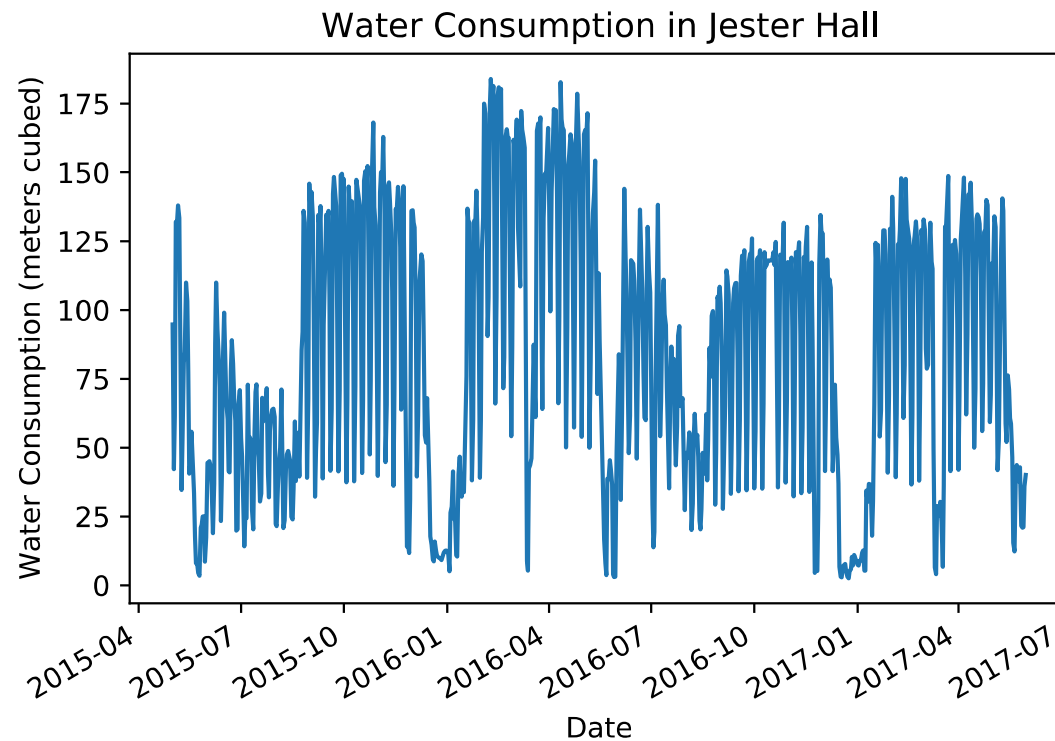
```
#Save Figure to PDF  
fig1.savefig('Figure 1 as PDF.pdf',dpi=600)  
  
#Save Figure to PNG  
fig1.savefig('Figure 1 as PNG.png',dpi=600)
```



- Save figures as a separate file to use for HW assignments, reports, etc.
- DO NOT take screenshots of Python figures and put them in HW or reports!

Save figures to PDF or PNG file

 Figure 1 as PNG.png	9/15/2020 2:58 PM	PNG File	580 KB
 Figure 1 as PDF.pdf	9/15/2020 2:58 PM	Adobe Acrobat D...	20 KB



Initial Explanatory Data Analysis

Uses the numpy package

```
#Numpy functions:

#mean
#If there are NaNs in your data, use np.nanmean
water_mean=np.mean(water_df['wF'])
print('The mean is ', water_mean)

#Median
#If there are NaNs in your data, use np.nanmedian
water_median=np.median(water_df['wF'])
print('The median is ', water_median)

#Variance
#If there are NaNs in your data, use np.nanvar
water_var=np.var(water_df['wF'])
print('The variance is ', water_var)

#Standard Deviation
#If there are NaNs in your data, use np.nanstd
water_std=np.std(water_df['wF'])
print('The standard deviation is ', water_std)
```

Uses the scipy package

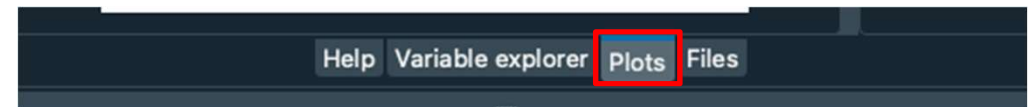
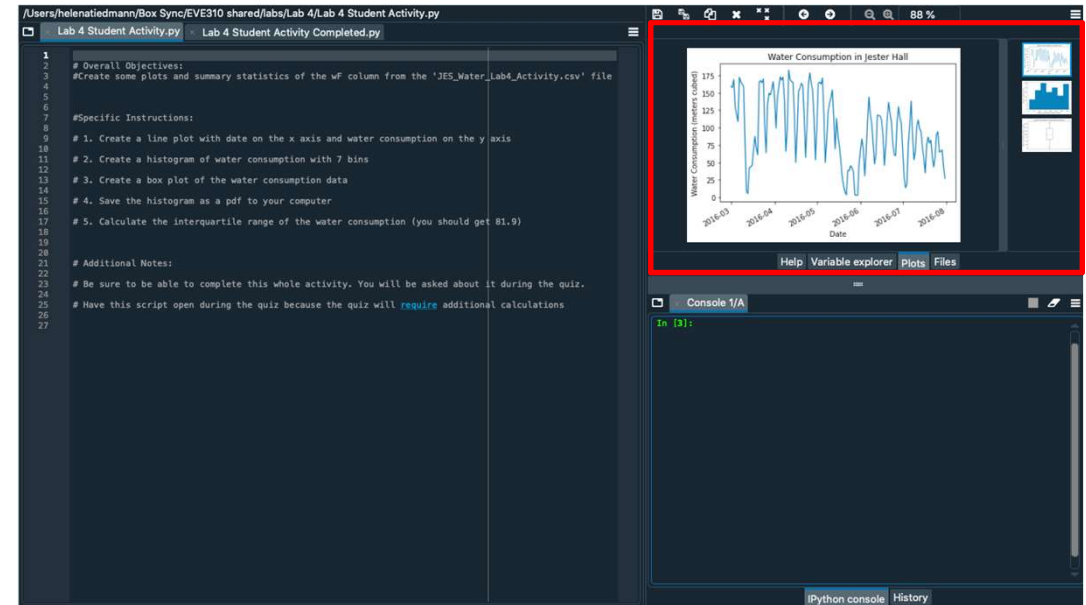
```
#SciPy Functions - remove NaNs first:
#Trimmed Mean with 10% of the data trimmed on the top and bottom
water_trimmed_mean=sc.trim_mean(water_df['wF'],.1)
print('The trimmed mean is ', water_trimmed_mean)

#Interquartile Range
water_iqr=sc.iqr(water_df['wF'])
print('The IQR is ', water_iqr)
```

Proportion of data to be trimmed from top and bottom (here, .1 means 10%)

A Note about Plotting using Spyder

- Each figure can show up as its own separate window
- Make sure close each figure between attempts at running the code so you don't get a bunch of figures on your screen!
- You can view all figures on the “Plots” tab



In-class activity

- Think about what steps you might need to take that are not explicitly asked of you in the assignment.
- Work through Lab 4 Student Activity
- **Make sure you can complete the activity before taking lab quiz 4. The quiz will ask you some questions about the activity.**

Learning objectives

- Use Scipy Package
- Convert to datetime format
- Creating graphics of data
 - Plot
 - Boxplot
 - Histogram
- Save Plots to PDF or Image
- Initial Exploratory Data Analysis